

Question Description:

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input

The first input line has two integers n and m : the number of cities and flights. The cities are numbered $1, 2, \dots, n$.

After this, there are m lines describing the flights. Each line has two integers a and b : there is a flight from city a to city b . All flights are one-way flights.

Output

First print an integer k : the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Test Case 1

INPUT (STDIN)

```
4 5
1 2
2 3
3 1
1 4
3 4
```

EXPECTED OUTPUT

```
1
4 1
```

Test Case 2

INPUT (STDIN)

```
4 5
1 3
2 1
1 2
2 4
1 4
```

EXPECTED OUTPUT

```
2
4 3
3 1
```

✓ Mandatory Test Cases

Test Case 1

KEYWORD

```
while(m--)
```

✓ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

1080

Test Case 3

NLOC

130

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 typedef struct {
6     int *head;
7     int *to;
8     int *nxt;
9     int edge_cnt;
10} Adj;
11
12void init_adj(Adj *a, int n, int max_edges) {
13    a->head = (int*)malloc((n + 1) * sizeof(int));
14    for (int i = 0; i <= n; i++) a->head[i] = -1;
15    a->to = (int*)malloc(max_edges * sizeof(int));
16    a->nxt = (int*)malloc(max_edges * sizeof(int));
17    a->edge_cnt = 0;
18}
19
20void add_dir_edge(Adj *a, int u, int v) {
21    a->to[a->edge_cnt] = v;
22    a->nxt[a->edge_cnt] = a->head[u];
23    a->head[u] = a->edge_cnt++;
24}
25
```

```

26void free_adj(Adj *a) {
27    free(a->head);
28    free(a->to);
29    free(a->nxt);
30}
31
32int n, m;
33int *order, order_cnt;
34int *visited;
35
36void dfs1(int u, Adj *g) {
37    visited[u] = 1;
38    for (int e = g->head[u]; e != -1; e = g->nxt[e]) {
39        int v = g->to[e];
40        if (!visited[v]) dfs1(v, g);
41    }
42    order[order_cnt++] = u;
43}
44
45int *scc;
46int scc_cnt;
47int *rep;
48

```

```

49void dfs2(int u, int c, Adj *rg) {
50    scc[u] = c;
51    for (int e = rg->head[u]; e != -1; e = rg->nxt[e]) {
52        int v = rg->to[e];
53        if (scc[v] == 0) dfs2(v, c, rg);
54    }
55}
56
57int *s_in, *s_out;
58int *matched_snk;
59int *vis_dag;
60
61int dfs_match(int u, Adj *dag, int *snk_visited) {
62    vis_dag[u] = 1;
63    if (s_out[u] == 0) {
64        if (!snk_visited[u]) {
65            snk_visited[u] = 1;
66            return u;
67        }
68    }
69    for (int e = dag->head[u]; e != -1; e = dag->nxt[e]) {
70        int v = dag->to[e];

```

```

71     if (!vis_dag[v]) {
72         int res = dfs_match(v, dag, snk_visited);
73         if (res != 0) return res;
74     }
75 }
76 return 0;
77}
78
79int main() {
80     if (scanf("%d %d", &n, &m) != 2) return 0;
81     Adj g, rg;
82     init_adj(&g, n, m + 5);
83     init_adj(&rg, n, m + 5);
84
85     int orig_m = m;
86     while(m--) {
87         int u, v;
88         if (scanf("%d %d", &u, &v) == 2) {
89             add_dir_edge(&g, u, v);
90             add_dir_edge(&rg, v, u);
91         }
92     }

```

```
93
94  order = (int*)malloc((n + 1) * sizeof(int));
95  order_cnt = 0;
96  visited = (int*)calloc(n + 1, sizeof(int));
97
98  for (int i = 1; i <= n; i++) {
99      if (!visited[i]) dfs1(i, &g);
100  }
101
```

main | CH.SC.U4CSE25225 | Tree23.c

Ln 202, Col 1 Spaces: 4 UTF-8 CRLF C

```
103  scc_cnt = 0;
104  rep = (int*)malloc((n + 1) * sizeof(int));
105
106  for (int i = n - 1; i >= 0; i--) {
107      int u = order[i];
108      if (scc[u] == 0) {
109          scc_cnt++;
110          rep[scc_cnt] = u;
111          dfs2(u, scc_cnt, &rg);
112      }
113  }
114
```

```

115     if (scc_cnt == 1) {
116         printf("0\n");
117         return 0;
118     }
119
120     Adj dag;
121     init_adj(&dag, scc_cnt, orig_m + 5);
122     s_in = (int*)calloc(scc_cnt + 1, sizeof(int));
123     s_out = (int*)calloc(scc_cnt + 1, sizeof(int));
124
125     for (int u = 1; u <= n; u++) {
126         for (int e = g.head[u]; e != -1; e = g.nxt[e]) {
127             int v = g.to[e];
128             if (scc[u] != scc[v]) {
129                 add_dir_edge(&dag, scc[u], scc[v]);
130                 s_out[scc[u]]++;
131                 s_in[scc[v]]++;
132             }
133         }
134     }
135
136     int *sources = (int*)malloc((scc_cnt + 1) * sizeof(int));

```

```

137 int num_sources = 0;
138 int *sinks = (int*)malloc((scc_cnt + 1) * sizeof(int));
139 int num_sinks = 0;
140
141 for (int i = 1; i <= scc_cnt; i++) {
142     if (s_in[i] == 0) sources[num_sources++] = i;
143     if (s_out[i] == 0) sinks[num_sinks++] = i;
144 }
145
146 vis_dag = (int*)malloc((scc_cnt + 1) * sizeof(int));
147 int *snk_visited = (int*)calloc(scc_cnt + 1, sizeof(int));
148
149 int *pair_src = (int*)malloc((scc_cnt + 1) * sizeof(int));
150 int *pair_snk = (int*)malloc((scc_cnt + 1) * sizeof(int));
151 int pair_cnt = 0;
152
153 for (int i = 0; i < num_sources; i++) {
154     for (int j = 1; j <= scc_cnt; j++) vis_dag[j] = 0;
155     int target_snk = dfs_match(sources[i], &dag, snk_visited);
156     if (target_snk != 0) {
157         pair_src[pair_cnt] = sources[i];
158         pair_snk[pair_cnt] = target_snk;

```

```

159         pair_cnt++;
160     }
161 }
162
163 int *unpaired_src = (int*)malloc((num_sources + 1) * sizeof(int));
164 int unp_src_cnt = 0;
165 int *src_is_paired = (int*)calloc(scc_cnt + 1, sizeof(int));
166 for (int i = 0; i < pair_cnt; i++) src_is_paired[pair_src[i]] = 1;
167 for (int i = 0; i < num_sources; i++) {
168     if (!src_is_paired[sources[i]]) unpaired_src[unp_src_cnt++] = sources[i];
169 }
170
171 int *unpaired_snk = (int*)malloc((num_sinks + 1) * sizeof(int));
172 int unp_snk_cnt = 0;
173 for (int i = 0; i < num_sinks; i++) {
174     if (!snk_visited[sinks[i]]) unpaired_snk[unp_snk_cnt++] = sinks[i];
175 }
176
177 int *all_ordered_src = (int*)malloc((num_sources + num_sinks + 5) * sizeof(int));
178 int *all_ordered_snk = (int*)malloc((num_sources + num_sinks + 5) * sizeof(int));
179
180 for (int i = 0; i < pair_cnt; i++) {

```

```

181     all_ordered_src[i] = pair_src[i];
182     all_ordered_snk[i] = pair_snk[i];
183 }
184
185 int cur_s = pair_cnt;
186 for (int i = 0; i < unp_src_cnt; i++) all_ordered_src[cur_s++] = unpaired_src[i];
187
188 int cur_t = pair_cnt;
189 for (int i = 0; i < unp_snk_cnt; i++) all_ordered_snk[cur_t++] = unpaired_snk[i];
190
191 int total_ans = num_sources > num_sinks ? num_sources : num_sinks;
192 printf("%d\n", total_ans);
193
194 for (int i = 0; i < total_ans; i++) {
195     int u_scc = all_ordered_snk[i % num_sinks];
196     int v_scc = all_ordered_src[(i + 1) % num_sources];
197     printf("%d %d\n", rep[u_scc], rep[v_scc]);
198 }
199
200 return 0;
201 }
202

```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

4 5

1 2

2 3

3 1

1 4

3 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree23.exe

1

4 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4 5

1 3

2 1

1 2

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree23.exe

2

4 1

3 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>